Lab 2: Design and implement C/C++ Program to find Minimum Cost Spanning Tree of a given connected undirected graph using Prim's algorithm

```
#include <stdio.h>

int main()
{
   int n;
   int graph[10][10];
   int visited[10] = {0};

   int edgeCount = 0;
   int totalCost = 0;

   printf("Enter number of vertices: ");
   scanf("%d", &n);

   printf("Enter cost matrix (999 if no edge or self loop):\n");
   for (int i = 1; i <= n; i++) {
      for (int j = 1; j <= n; j++) {
         scanf("%d", &graph[i][j]);

      }
   }

   // Start from vertex 1
   visited[1] = 1;

   printf("\nEdges in Minimum Spanning Tree:\n");

   while (edgeCount < n- 1) {
      int minCost = 999;
      int fromVertex = -1;
      int toVertex = -1;

      // Find smallest edge from visited → unvisited
      for (int i = 1; i <= n; i++) {
         if (visited[i] == 1) {
            for (int j = 1; j <= n; j++) {
               if (visited[j] == 0 && graph[i][j] < minCost) {
                  minCost = graph[i][j];
                  fromVertex = i;
                  toVertex=j;
            }
             }
         }
      }

      printf("Edge %d: (%d -> %d) cost = %d\n", edgeCount + 1, fromVertex, toVertex, minCost);
```

```
        totalCost += minCost;
        visited[toVertex] = 1;  // Add new vertex to MST
        edgeCount++;
    }

    printf("\nTotal Minimum Cost = %d\n", totalCost);

    return 0;
}
```

OUTPUT

```
Enter number of vertices: 6
Enter cost matrix (999 if no edge or self loop):
999 3 999 999 6 5
3 999 1 999 999 4
999 1 999 6 999 4
999 999 6 999 8 5
6 999 999 8 999 2
5 4 4 5 2 999

Edges in Minimum Spanning Tree:
Edge 1: (1 -> 2) cost = 3
Edge 2: (2 -> 3) cost = 1
Edge 3: (2 -> 6) cost = 4
Edge 4: (6 -> 5) cost = 2
Edge 5: (6 -> 4) cost = 5

Total Minimum Cost = 15
```